

Reviewer Pack — Algebraic K-Theory Rank Bounds for Resolution Proof Width

Entry 5adbb526b84d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 15:08:56 UTC

Algebraic K-Theory Rank Bounds for Resolution Proof Width

Entry ID: 5adbb526b84d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 15:08:56 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory and Representation Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For every SAT instance F , the rank of its algebraic K-theory $A(K_0(F))$ is upper-bounded by a function $g(n) = \Theta(\log n)$, where n is the number of variables in F .’, ‘Equivalently, for any resolution refutation of an unsatisfiable 3-CNF formula F with m clauses and n variables, the width w^* of its resolution proof tree satisfies $w^* \leq 2^n / \log 2 * g(n)$.’, ‘Consequently, if there exists a SAT instance F such that $\text{rank}(A(K_0(F))) > \Theta(\log n)$, then the resolution proof for F must have an exponential number of steps.’]

Rationale (proposer’s reasoning):

[‘Algebraic K-theory is a branch of algebraic geometry that studies topological invariants of rings, which could potentially reveal structural properties of complex computational objects like resolution proofs.’, ‘Previous work has shown connections between the rank of Brauer groups and circuit complexity. Extending this to algebraic K-theory may provide new insights into the nature of resolution proofs and their complexity.’]

Taxonomy category: Algebraic_K_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): aac2ab927e4f42c6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 24 out of 30 random seeds, the correlation coefficient (r) between $\text{rank}(A(K_0(F)))$ and w^* is greater than or equal to 0.7, with no seed showing a $\text{rank}(A(K_0(F))) > \Theta(\log n)$, where $w^* \leq 2^n / \log 2 * g(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "algebraic K-theory" AND "resolution proof complexity" - "proof width" IN Alg-K-Theory AND Complexity-Theory - "rank bounds" IN Algebraic-K-Theory AND Resolution-Proof-Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n, m):
        clauses = []
        variables = [f"x{i}" for i in range(1, n+1)]
        neg_variables = [f"~x{i}" for i in range(1, n+1)]

        for _ in range(m):
            clause = random.sample(variables + neg_variables, 3)
            clauses.append(clause)

        return clauses

    def dpll_solve(clauses, assignment={}):
        if not clauses:
            return True
        if any(all(var not in assignment or assignment[var] != val for var, val in clause) for clause in clauses):
            return False

        var = next((var for var in variables if var not in assignment), None)
        if var is None:
            return True

        positive_var = var
        negative_var = f"~{var}"

        if dpll_solve([c for c in clauses if all(var not in c or assignment[var] != val for var, val in c)]):
            return True
```

```

    if dpll_solve([c for c in clauses if all(var not in c or assignment[var] != val for var, v
        return True

    return False

def algebraic_k_theory_rank(clauses):
    # Placeholder function to simulate the computation
    # Replace with actual algorithm for computing K-theory rank
    return random.randint(1, 10)

n = random.randint(5, 40)
m = random.randint(n * 2, n * 3)
clauses = generate_3cnf(n, m)

k_theory_rank = algebraic_k_theory_rank(clauses)
width = dpll_solve(clauses) # Assuming DPLL returns the number of steps

return {
    "metric_name": "Correlation between K-theory rank and resolution width",
    "metric_value": k_theory_rank * width,
    "instances_tested": 1,
    "conjecture_holds": False,
    "counterexample": "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2bcf82fb.py", line 77, in <module>

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2bcf82fb.py", line 62, in run_trial
    width = dpll_solve(clauses) # Assuming DPLL returns the number of steps
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2bcf82fb.py", line 35, in dpll_solve
    if any(all(var not in assignment or assignment[var] != val for var, val in clause) for clause in clauses):
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2bcf82fb.py", line 35, in <genexpr>
    if any(all(var not in assignment or assignment[var] != val for var, val in clause) for clause in clauses):
    ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2bcf82fb.py", line 35, in <genexpr>
    if any(all(var not in assignment or assignment[var] != val for var, val in clause) for clause in clauses):
    ~~~~~
```

8. Critic adversarial review

Critic reasoning:

9. Final verdict & safety rail

Reasoning:

11. Audit log (LLM calls)

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12629
2	preregistration	ollama_remote	glm4:latest	0	0	6027
3	novelty	ollama_remote	glm4:latest	0	0	4681
4	novelty	ollama_remote	glm4:latest	0	0	5253
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19502
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10896
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9570
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9155
9	judge	ollama_remote	glm4:latest	0	0	9624

(full prompt+response transcripts available in [research/audit/5adbb526b84d.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5adbb526b84d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5adbb526b84d.tar.gz` (if generated)